Comparative Study of Lightweight Deep Learning Architectures for Responsible Edge AI Deployment in Semiconductor Defect Classification

A Project Report Submitted

in Partial Fulfilment of the Requirements

for the Degree of

Master of Technology

in

Computer Science and Engineering with Specialization

in

Bigdata & Machine Learning

by

PRASAD MALAI

(Roll No. 2023MCS320002)

to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

KOTTAYAM-686635, INDIA

May 2026

DECLARATION

I, **Prasad Malai** (**Roll No: 2023MCS320002**]), hereby declare that, this report entitled **"Comparative Study of Lightweight Deep Learning Architectures for Responsible Edge AI Deployment in Semiconductor Defect Classification"** submitted to Indian Institute of Information Technology Kottayam towards partial requirement of **Master of Technology** in **Computer Science and Engineering with Specialization in Big Data & Machine Learning** is an original work carried out by me under the supervision of **Dr. Ebin Deni Raj** and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. I have sincerely tried to uphold the academic ethics and honesty. Whenever an external information or statement or result is used then, that have been duly acknowledged and cited.

Kottayam-686635 **[Prasad Malai]**

May 2026

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Comparative Study of Lightweight Deep Learning Architectures for Responsible Edge AI Deployment in Semiconductor Defect Classification"** submitted by **Prasad Malai** (**Roll No: 2023MCS320002**) to Indian Institute of Information Technology Kottayam towards partial requirement of **Master of Technology** in **Computer Science and Engineering with Specialization in Big Data & Machine Learning** has been carried out by him under my supervision and that it has not been submitted elsewhere for the award of any degree.

Kottayam-686635 (Dr. Ebin Deni Raj)

May 2026 Project Supervisor

ABSTRACT

Semiconductor defect classification is crucial in ensuring product quality and yield in modern semiconductor manufacturing processes. This work explores the feasibility of deploying edge AI–based solutions for accurate and efficient defect classification, addressing the challenges of limited computational resources and real-time inference requirements. A comparative study of lightweight convolutional neural network architectures, including MobileNetV2, ShuffleNet, and ResNet18, is conducted for multi-class defect classification.

The models are trained and subsequently optimized for edge deployment using techniques such as model quantization and ONNX-based conversion, resulting in reduced latency and enhanced cross-platform portability. Model performance is evaluated in terms of classification accuracy, inference efficiency, and suitability for real-time edge deployment. To enhance interpretability, Gradient-weighted Class Activation Mapping (Grad-CAM) is employed to visualize the regions influencing model predictions. Quantitative evaluation of the Grad-CAM outputs is performed using metrics such as Mean Intersection over Union (IoU) and Activation Ratio, enabling assessment of localization quality.

Experimental results demonstrate that ResNet18 achieves superior localization performance due to its deeper architecture, while MobileNetV2 and ShuffleNet offer a favorable trade-off between computational efficiency and accuracy. The findings highlight the importance of balancing model complexity and interpretability for responsible edge AI deployment in semiconductor defect classification.

Keywords:

Edge Computing, Semiconductor Inspection, Scanning Electron Microscopy (SEM), Raspberry Pi, Quantization, Explainable AI, Grad-CAM

Contents

List of Figures **x**

List of Tables **xi**

1 Introduction **1**

1.1 Background of Defect Classes 2

1.1.1 Semiconductor Manufacturing Overview 2

1.1.2 What are Wafer Defects and Why They Occur 2

1.1.3 Types of Defects 2

1.1.4 Traditional Inspection approaches 4

1.2 Importance of defect classification 5

1.2.1 Quality Improvement 6

1.2.2 Yield Enhancement 6

1.2.3 Process Automation 7

1.2.4 Cost Reduction 7

1.2.5 Data-Driven Manufacturing 8

1.3 Limitations and Motivation 8

1.3.1 Limitation with current approach 8

1.3.2 Motivation for Using Edge AI 9

2 Literature Overview **10**

2.1 Initial related Work 12

2.1.1 Deep Learning Model for Automated Visual Inspection of Electronic Boards(PCBs) 12

2.1.2 Effective and Lightweight Surface Defect Detection via Enhanced Deep Learning Models 13

2.2 Recent related work 16

2.2.1 Summary of Existing Research Approaches 16

2.3 Research gap 17

2.3.1 Accuracy Priority Over Deployability 17

2.3.2 Limited Edge AI Adoption 17

2.3.3 Very few studies address actual industry needs: 18

2.4 Conclusion 18

3 Problem Statement & Research overview **20**

3.1 Problem Statement 20

3.2 Objective 21

3.2.1 Performance Benchmark Report 21

3.2.2 Edge Deployment Feasibility Report 22

3.2.3 Responsible AI Assessment 23

3.2.4 Design Guidelines for Edge vs Cloud Deployment . . . 24

4 System Methodology **26**

4.1 Data collection and Preparation 27

4.1.1 Data Collection Approach 27

4.1.2 Defect Classes Considered 29
4.1.3 Data Challenges 30
4.1.4 Data Pre-processing 31
4.1.5 Data Augmentation 33
4.1.6 Dataset Split 34
4.1.7 Summary 35
4.2 Model Architecture and Training Methodology 36
4.2.1 Edge Deployable Model Selection Criteria 38
4.2.2 Model 1: MobileNetV2 38
4.2.3 Why MobilnetV2 is selected 39
4.2.4 Model 2: ShuffleNet 40
4.2.5 Why Shufflenet is selected 40
4.2.6 Model3:ResNet18 41
4.2.7 Why ResNet18 is selected 41
4.3 Training Methodology 42
4.3.1 Why Edge Optimization is Required 42
4.3.2 Quantization 43
4.3.3 Why ONNX is Needed 44
4.3.4 Quantization + ONNX Together 45
4.4 Grad-CAM Usage in the Proposed Semiconductor Defect Classification System 46
4.4.1 How Grad-CAM Was Used in This Project 46
4.4.2 How It Was Used for Each Defect Class 47
4.4.3 Interpretation of Model Failures through Grad-CAM Visualization 48

5 Result analysis and discussion **49**

5.1 Experimental setup 49

5.2 Training Performance analysis 52

5.2.1 Training overview for Mobilnet V2 52

5.2.2 Training overview for Shufflenet 53

5.2.3 Training overview for Resnet18 54

5.3 Edge inference result over Raspberry Pi4 55

5.3.1 Edge inference result for MobilNetV2 55

5.3.2 Edge inference result for ShuffleNet 55

5.3.3 Edge inference result for ResNet18 56

5.3.4 Quantitative Model Comparison 56

5.4 Grad-CAM Qualitative and quantitative analysis 57

5.4.1 Grad-CAM visualization for MobilenetV2 57

5.4.2 Grad-CAM visualization for ShuffleNet 58

5.4.3 Grad-CAM visualization for ResNet18 58

5.5 Observations & Reasoning: 59

5.5.1 Correlation Between Classification Performance and Localization 59

5.5.2 ResNet18 – High Accuracy, Strong Localization, Higher Cost 59

5.5.3 MobileNetV2 – Balanced Trade-off 60

5.5.4 ShuffleNet – Efficient but Limited Learning Capacity . 61

5.5.5 Edge Deployment Trade-off Analysis 61

5.5.6 Grad-CAM Analysis for Correct and Incorrect Predictions 62

5.5.7 Final comparative insight 62

6 Future work **64**

7 Conclusion **65**

Bibliography **67**

List of Figures

4.1 Semiconductor defects 30
4.2 Dataset Overview and Statistical Analysis summary 35
4.3 Architectural Overview 36
4.4 End to End defect classification Pipeline 36

5.1 MobilnetV2 Training Overview 52
5.2 ShuffleNet Training Overview 53
5.3 ResNet18 Training Overview 54
5.4 Edge inference result for MobilNetV2 55
5.5 Edge inference result for ShuffleNet 55
5.6 Edge inference result for ResNet18 56
5.7 Grad-Cam over MobilNetV2 57
5.8 Grad-Cam over ShuffleNet 58
5.9 Grad-Cam over ResNet18 58

List of Tables

5.1 Training Parameters for MobileNetV2 50
5.2 Training Parameters for ShuffleNet 51
5.3 Training Parameters for ResNet18 52
5.4 Classification Performance of Selected Models 56
5.5 Edge Inference Performance of Selected Models 56
5.6 Quantitative Grad-CAM Comparison of Selected Models . . . 57

Chapter 1

Introduction

In recent years, the semiconductor industry has experienced significant growth driven by advancements in manufacturing technologies and the increasing demand for highly efficient, reliable, and high-performance integrated circuits. In modern semiconductor fabrication, defect detection and classification play a critical role in maintaining production efficiency and ensuring product reliability. Despite advancements in inspection systems (automatic and semi-automatic), the reduced size of integrated circuits and the increasingly complex nature of defect patterns continue to pose significant challenges to these approaches. Although deep learning techniques have demonstrated promising results in automated defect analysis, their deployment in real-world environments is often constrained by high computational requirements and limited interoperability across platforms. This has led to increasing interest in lightweight, edge-deployable solutions that can operate efficiently in resource-constrained and heterogeneous environments.

1.1 Background of Defect Classes

1.1.1 Semiconductor Manufacturing Overview

Semiconductor manufacturing is a highly complex, multi-stage process used to fabricate integrated circuits (ICs) on silicon wafers. The manufacturing process involves repeated cycles of lithography, deposition, etching, doping, and polishing, all performed with nanometer-level precision. Each wafer undergoes numerous processing steps in a clean room environment. Even minor variations in process conditions can introduce defects that adversely affect circuit functionality.

1.1.2 What are Wafer Defects and Why They Occur

Wafer defects refer to unwanted irregularities or anomalies that occur on the surface or within the semiconductor wafer during fabrication. These defects can arise due to:

- Particle Containment in clean rooms
- Equipment malfunction or calibration errors
- Process variation (temperature, pressure, timing)
- Material imperfections in silicon wafers

1.1.3 Types of Defects

- **Structural Defects:** These are physical damages to the wafer surface.

  Examples:

  - Scratches
  - Cracks
  - Breaks in the wafer surface

  Characteristics:

  - Often caused by handling or by mechanical stress
  - Visible in imaging systems
  - May affect entire circuit functionality

- **Contamination Defects:** These are foreign particles or residues on the wafer surface. Examples:

  - Dust particles
  - Chemical residues
  - Metallic contamination

  Characteristics:

  - Usually random in distribution
  - Can be very small but still be impactful
  - Often introduced during cleaning or pre/post processing steps

- **Pattern-Related Defects:** These defects arise when the intended geometric layout of the integrated circuit is incorrectly formed, leading to deviations from the designed pattern. Examples:

  - Bridging (unwanted connection between circuit lines)

- Open circuits (break in expected pattern)
- Misalignment of patterns
- Missing or extra pattern features

Characteristics:

- Occur during lithography or etching steps
- Directly affect circuit behavior
- Harder to detect visually compared to other structural defects

1.1.4 Traditional Inspection approaches

- **Optical Inspection Method:** This approach uses high-resolution imaging systems to examine semiconductor wafers. The captured images are analyzed to identify irregularities such as scratches, contamination, or pattern deviations during the manufacturing process.

  How they work:

  - Capture wafer images
  - Compare with reference patterns or neighboring dies
  - Flag deviations as defects

  Limitations:

  - High cost
  - Sensitive to noise

  - Struggles with complex patterns in advanced nodes
  - Requires post-processing interpretation

- **Manual Classification:** Here human experts visually inspect wafer maps or microscope images to identify defect types. Limitations:
  - Time-consuming
  - Subjective (depends on expert experience)
  - Not scalable for high-volume production
  - Inconsistent labeling across engineers

Due to the limitations of traditional inspection methods, there has been a significant shift toward automated defect classification using machine learning and deep learning techniques. These approaches aim to improve accuracy, scalability, and real-time decision-making in semiconductor manufacturing environments.

1.2 Importance of defect classification

Defect classification is a key component of semiconductor manufacturing, as it influences product quality, overall yield, and the effectiveness of automated production processes. As device geometries shrink and fabrication complexity increases, accurate and efficient defect analysis becomes essential.

1.2.1 Quality Improvement

Defect classification ensures that only wafers meeting stringent quality standards proceed to packaging and deployment. By identifying and categorizing defects (such as scratches, contamination, or pattern irregularities), manufacturers can:

- Detect faulty dies early in the production cycle
- Prevent defective chips from reaching customers
- Maintain reliability in high-performance applications (e.g., automotive, medical, aerospace systems)

This is especially crucial in safety-critical domains like automotive embedded systems, where even minor defects can lead to system failures.

1.2.2 Yield Enhancement

Yield refers to the percentage of functional chips produced from a wafer. Defect classification directly contributes to yield improvement by:

- Identifying recurring defect patterns linked to specific process steps
- Enabling root cause analysis (e.g., lithography errors, etching issues)
- Reducing scrap rates and rework costs

By analyzing defect trends over time, manufacturers can optimize fabrication parameters and significantly increase the number of usable chips per wafer, leading to higher profitability.

1.2.3 Process Automation

Traditional defect inspection methods rely heavily on manual review, which is:

- Time-consuming
- Subject to human error
- Not scalable for high-volume production

Automated defect classification systems, especially those based on deep learning, enable:

- Real-time inspection and classification of wafer defects
- Consistent and objective decision-making
- Integration with smart manufacturing pipelines (Industry 4.0)

Automation reduces dependency on human experts and allows fabs to operate efficiently at scale.

1.2.4 Cost Reduction

Accurate defect classification minimizes:

- Material wastage
- Rework and repair costs
- Downtime due to unidentified process issues

Early detection and classification prevent defects from propagating through multiple stages, saving significant operational costs.

1.2.5 Data-Driven Manufacturing

Defect classification generates valuable data that can be used for:

- Predictive maintenance of equipment
- Statistical process control (SPC)
- Continuous process improvement

This enables a shift from reactive to proactive manufacturing strategies.

1.3 Limitations and Motivation

1.3.1 Limitation with current approach

- High-performing deep learning models (e.g.InceptionNet (GoogLeNet), DenseNet) are computationally expensive
- Dependence on cloud-based processing, leading to latency and bandwidth issues
- Limited suitability for real-time defect detection in high-throughput manufacturing
- Black-box nature of models reduces trust in critical manufacturing decisions
- Poor generalization across fabrication environments
- Heavy reliance on large labeled datasets
- Traditional methods lack robustness, while deep models lack efficiency

1.3.2 Motivation for Using Edge AI

- Enables real-time defect classification directly on inspection machines
- Reduces latency by avoiding cloud communication
- Minimizes network bandwidth usage (important for high-resolution wafer images)
- Improve data privacy and security (sensitive manufacturing data stays on-site)
- Supports scalable deployment across multiple inspection units in fabs
- Allows integration with Industry 4.0 smart manufacturing pipelines

In conclusion, as semiconductor manufacturing moves towards fully automated and intelligent systems, the role of efficient and explainable defect classification becomes increasingly important. The convergence of Edge AI and interpretable deep learning offers a promising direction to achieve real-time, reliable, and scalable solutions. This work is driven by the vision of enabling practical, trustworthy edge AI systems that can be seamlessly integrated into next-generation smart manufacturing environments.

Chapter 2

Literature Overview

The semiconductor manufacturing industry is a continuously evolving sector in which defect detection and classification play a crucial role. Accurate and cost-effective inspection systems that can identify and classify semiconductor defects at an early stage of the manufacturing process are essential for improving productivity, yield, and revenue in this market sector.

In the early stages of the industry, when manufacturing processes were less automated, human operators performed manual inspections because defects appearing on wafers were relatively larger and easier to identify. However, as semiconductor feature sizes continued to shrink, defects also became smaller and more complex, making manual inspection increasingly challenging. This created the need for alternative and more reliable inspection methods. The growth of computer vision techniques and advancements in microscopy technologies significantly contributed to addressing this challenge.

The resolution limit for defect detection is defined as the minimum distance at which two separate structures can be distinguished as distinct ob-

jects. For Scanning Electron Microscopy (SEM), with appropriate settings, the resolution can reach approximately 1–10 nm. Such improvements in imaging resolution allow significantly more information to be extracted from wafer images, leading to enhanced performance in defect detection and classification tasks.

Initially, machine learning (ML)-based approaches appeared promising for semiconductor defect classification. These methods included algorithms such as Support Vector Machines (SVM), Decision Trees (DT), and K-Nearest Neighbors (K-NN). However, traditional ML approaches require large volumes of labeled training data and careful manual feature engineering. Since model performance heavily depends on the quality of selected features, there was a growing need for more advanced alternatives. One such alternative is Deep Learning (DL).

Deep learning provides notable benefits compared to traditional machine learning techniques, particularly for complex tasks in this domain. Unlike ML methods, deep learning models can automatically learn hierarchical feature representations directly from raw image data, reducing the need for manual feature extraction. They are highly effective in handling complex patterns, subtle defects, and large-scale datasets. Furthermore, Convolutional Neural Networks (CNN) have shown superior performance in image classification tasks, making them particularly suitable for semiconductor defect inspection.

2.1 Initial related Work

2.1.1 Deep Learning Model for Automated Visual Inspection of Electronic Boards(PCBs)

In this context, José Antonio Lara-Chávez, Carlos Avilés-Cruz, and Miguel Magos-Rivera proposed a deep learning framework titled Deep Learning Model for Automated Visual Inspection of Electronic Boards, published in the Journal of Intelligent Manufacturing. Although the study focuses on Printed Circuit Board (PCB) assembly inspection, its methodology is highly relevant to semiconductor defect classification, as both applications involve image-based quality inspection of miniature components and subtle defect patterns.

The proposed model consists of three sequential stages: segmentation, feature extraction, and classification. In the first stage, Mask R-CNN was employed to detect and isolate the PCB region from the input image, ensuring that irrelevant background information does not affect the classification process. During the second phase, a multi-level feature extraction mechanism is employed, here parallel CNN branches are designed to process coarse, medium, and fine-grained information, similar to how the human visual system interprets details. This design captures large structural information, medium-level patterns, and fine defect details simultaneously. Finally, a fully connected neural network was used to classify the boards into one normal class and five defect classes.

The model trained and evaluated using a proprietary dataset containing 2,376 RGB images that represents various PCB assembly defects such as

missing integrated circuits, reversed polarity components, incorrect resistor values, and misplaced devices. Experimental results reported 100% classification accuracy during training and evaluation, while real-time implementation tests achieved 98.33% practical accuracy.

This work demonstrates that CNN-based hierarchical feature extraction is highly effective for detecting subtle manufacturing defects under real-world conditions. The study also confirms the potential of automated inspection systems to replace manual quality control processes.

However, despite its strong performance, the proposed framework relies on computationally intensive architectures such as Mask R-CNN and multiple parallel CNN branches, requiring high-end GPU hardware for training and deployment. In addition, the work does not report model interpretability, which is important in safety-critical industrial environments where engineers must verify model decisions. Furthermore, the study focuses on PCB assembly defects rather than semiconductor wafer defect patterns.

2.1.2 Effective and Lightweight Surface Defect Detection via Enhanced Deep Learning Models

Surface defect detection plays a crucial role in industrial quality assurance, particularly in manufacturing sectors such as steel, semiconductors, electronics, and automotive production. Conventional defect inspection methods based on manual observation or classical image processing often suffer from low accuracy, poor adaptability to varying conditions, and limited robustness when defect patterns become subtle or complex. These limitations have motivated industries to adopt machine learning and deep learning-based

automated inspection systems. In a recent study, an enhanced defect detection framework was proposed by integrating EfficientNet-B0 with YOLOv8 for steel surface defect detection. The objective of the work was to improve detection efficiency while reducing computational resource consumption, thereby enabling practical deployment in industrial environments. The study uses widely recognized NEU-DET dataset, developed by Kechen Song et al., containing 1,800 grayscale images across six common steel surface defect categories:

- Crazing
- Inclusion
- Patches
- Pitted surface
- Rolled-in scale
- Scratches

To improve model robustness, the authors applied data augmentation techniques such as:

- Rotation
- Flipping
- Scaling
- Contrast adjustment

These methods increased defect variability and reduced overfitting during training. The proposed architecture replaces the original YOLOv8 backbone with EfficientNet-B0. Input images first pass through Mobile Bottleneck Convolution (MBConv) blocks for efficient feature extraction. Multi-scale feature fusion and detection heads are then used to identify defects of varying sizes. This design significantly improves detection of small and low-resolution defects while maintaining a lightweight structure. The model was trained for 300 epochs and compared against baseline YOLOv8 trained for 200 epochs.

Experimental results demonstrated substantial improvement:

- Baseline YOLOv8 mAP@0.5 = 0.964
- Improved YOLOv8 + EfficientNet-B0 mAP@0.5 = 0.993

Class-wise performance also improved considerably, particularly for difficult classes such as:

- Crazing (+8.7 percent)
- Pitted surface (+5.2 percent)

These findings indicate that lightweight backbone optimization can significantly enhance defect detection performance without increasing computational burden. This work is highly relevant to the present research because it demonstrates that combining lightweight CNN architectures with efficient detection frameworks can provide high accuracy and practical deployability. Similar challenges exist in semiconductor defect classification, where defect patterns are often small, low-contrast, and highly varied.

2.2 Recent related work

A recent study investigated the use of Vision Transformers (ViTs), specifically the DinoV2 model, for automated semiconductor defect classification using Scanning Electron Microscope (SEM) images. The researchers used more than 7,400 real defect images collected from two wafer inspection layers covering 11 defect categories. Initially, pretrained DinoV2 features combined with a k-nearest neighbors classifier achieved strong results, showing effective defect representation even without task-specific training. Further improvement was obtained through transfer learning, where the model reached over 90 percent classification accuracy using only a small number of labeled samples per defect class. Semi-supervised learning with pseudo-labeling also provided additional gains. The study demonstrated that transformer-based models are highly effective for semiconductor defect inspection, particularly in scenarios with limited labeled data, though their higher computational complexity may limit deployment on edge devices.

2.2.1 Summary of Existing Research Approaches

Existing Methods Focused Mainly On:

- CNN-based defect classification with high accuracy using centralized GPUs/cloud systems.
- Transformer and Generative AI frameworks improving feature learning for subtle defects.
- Ensemble and hybrid models increasing classification performance.

- Machine learning surveys showing strong progress in wafer defect recognition.
- Lightweight CNN architecture like Squeeze-Net / low-complexity CNN for faster classification.

2.3 Research gap

Key Research Gaps identified include:

2.3.1 Accuracy Priority Over Deployability

Most prior work focuses on maximizing classification accuracy, but:

- High-parameter models need GPU servers
- High inference latency in production lines
- Difficult to deploy directly on inspection machines
- Increased hardware cost

2.3.2 Limited Edge AI Adoption

Most models are trained/tested in:

- Cloud environments
- Workstations
- Kaggle / lab systems

Very few studies address:

- On-device inference
- Low-power processors
- Industrial embedded boards
- Local real-time inspection nodes

2.3.3 Very few studies address actual industry needs:

- Lack of Explainability / Trust
- Many deep learning systems behave as black boxes.
- In semiconductor fabrication, engineers need: Why was the wafer is rejected?, Which is the defect region?, Influenced the decision?
- Is the model focusing on actual scratch or noise?

2.4 Conclusion

Although prior studies have achieved promising accuracy in semiconductor defect classification using CNNs, transformers, and ensemble learning, limited attention has been given to deployable, low-latency, and interpretable solutions suitable for real-time industrial environments. Most existing approaches rely on computationally intensive centralized systems and provide limited transparency in decision-making. Therefore, there is a need for an

efficient Edge AI-based framework using lightweight deep learning models integrated with explainability techniques to enable practical, trustworthy, and scalable semiconductor defect classification.

Chapter 3

Problem Statement & Research overview

3.1 Problem Statement

Semiconductor fabrication facilities generate massive volumes of high-resolution wafer and die inspection images, where early and accurate identification of defects is essential for maintaining yield, reliability, and cost efficiency. While automated optical inspection systems are widely used for defect detection, the subsequent defect classification stage frequently relies on centralized cloud servers or manual expert analysis. This dependency creates several bottlenecks, such as communication latency, network bandwidth limitations, privacy concerns related to sensitive manufacturing data, and increased operational costs. Additionally, these issues hinder real-time inspection in semiconductor manufacturing lines. Hence, there is a critical need for an Edge-AI-enabled defect classification framework that can execute directly

on low-power edge hardware with limited memory and computational resources. The proposed system should deliver high classification performance with reduced latency and optimized resource utilization, enabling practical and scalable deployment in semiconductor fabrication environments.

3.2 Objective

3.2.1 Performance Benchmark Report

To systematically compare selected deep learning models for semiconductor defect classification using multiple evaluation metrics such as Accuracy, Precision, Recall, F1-score, inference latency, and memory usage. In many research works, models are compared only using classification accuracy. However, for real-world semiconductor manufacturing systems, high accuracy alone is insufficient. A model must also be fast, lightweight, and efficient.

Therefore, this objective focuses on creating a benchmark report that evaluates each candidate model (such as MobileNet, ResNet, ShuffleNet, etc.) using:

- Accuracy: Overall percentage of correctly classified defect images.
- Precision: How many predicted defects are correctly identified.
- Recall: Ability to detect all actual defects.
- F1-score: Balanced metric combining precision and recall.
- Inference Latency: Time required to classify one image.

- Memory Consumption: RAM/storage required during deployment.
- Model Size: Disk space occupied by trained model.
- Computational Complexity: FLOPs or parameter count.

Expected Outcome:

A comparative table ranking all models and identifying the best trade-off between performance and efficiency.

3.2.2 Edge Deployment Feasibility Report

To evaluate the practical suitability of the trained models for deployment on low-resource edge devices, including platforms like the Raspberry Pi, using quantitative metrics. Although many deep learning models perform well on GPUs or cloud servers, they may fail on edge hardware because of:

- Low CPU power
- Limited RAM
- Thermal constraints
- Power limitations
- Slower storage access

Hence, following measurements are taken after deploying the model over Raspberry Pi4

- Real inference time per image

- CPU utilization
- RAM consumption
- Frames per second (FPS)
- Accuracy drop after optimization (quantization/pruning)

3.2.3 Responsible AI Assessment

To ensure the selected AI model is transparent, unbiased, fair, and trustworthy before deployment in semiconductor manufacturing.

Industries increasingly require AI systems to be explainable and reliable. If the model misclassified defects, it may cause:

- Yield loss
- Wrong rejection of good wafers
- Missed critical defects
- Financial loss

Hence, this objective evaluates:

- Explainability Analysis: Grad-CAM and feature activation map
- Bias Evaluation: Across Defect Classes
- Fairness Assessment: Ensure all defect categories receive balanced treatment.

3.2.4 Design Guidelines for Edge vs Cloud Deployment

To develop recommendations that identify appropriate scenarios for deploying semiconductor defect classification on edge platforms versus cloud-based systems.

Edge Deployment Suitable When:

- Real-time response needed
- Low latency is essential
- Internet unavailability or limited connectivity
- Data privacy
- High-volume continuous inspection
  Examples:
  Instant reject/accept decision on production lines
  Real time quality monitoring etc.

Cloud Deployment Suitable When:

- Heavy model training required
- Large storage needed
- Historical analytics

- Cross-fab monitoring
- Retraining from global datasets

Example use cases:

Weekly yield trend analysis

Model retraining etc.

Chapter 4

System Methodology

This chapter presents the methodology adopted for the development of the proposed Edge-AI-based semiconductor defect classification system. The objective of the system is to accurately classify semiconductor defects while satisfying practical deployment constraints such as low latency, reduced memory usage, and computational efficiency on resource-constrained edge devices.

The overall approach includes preparing the dataset, applying image preprocessing and augmentation techniques, selecting suitable lightweight models, performing transfer learning-based training, evaluating performance, optimizing the models, and assessing their feasibility for deployment on edge devices. In addition, explainability and responsible AI assessment techniques are incorporated to ensure reliable and trustworthy model predictions.

The complete workflow of the proposed system is illustrated in Figures 4.3 and 4.4

4.1 Data collection and Preparation

The performance of any deep learning–based defect classification system is highly dependent on the quality, diversity, and representativeness of the training dataset. In semiconductor manufacturing, obtaining large-scale labeled defect datasets is a major challenge due to intellectual property restrictions, confidentiality agreements, limited public release of fab inspection data, and the high cost of expert annotation. As a result, publicly available semiconductor image datasets often contain limited defect categories, low sample counts, or incomplete metadata.

To address these practical constraints, a hybrid dataset generation strategy was adopted in this work by combining publicly available semiconductor defect images with synthetically generated defect samples. This approach enabled the creation of a balanced and sufficiently diverse dataset suitable for training and evaluating lightweight deep learning models under realistic research constraints.

4.1.1 Data Collection Approach

Since access to proprietary industrial semiconductor datasets was not available, the dataset used in this research was developed from two complementary sources:

Public Semiconductor Defect Datasets

Publicly available semiconductor inspection datasets were explored to obtain real defect image samples representing defects observed during wafer or die

inspection. These datasets offer realistic representations of defect textures, process-induced noise, and manufacturing variations that are challenging to replicate manually. Publicly available datasets were primarily utilized to capture the following characteristics:

- Authentic inspection image properties
- Practical background noise conditions
- Irregular and complex defect boundaries
- Real-world imaging environments
- Variations in illumination and contrast

Synthetic Defect Image Generation

Due to the limited availability of publicly labeled data, a synthetic dataset was generated to supplement the existing dataset. Artificial defect patterns were created using geometric structures, shape distortions, line interruptions, cracks, and conductive path abnormalities resembling semiconductor layout defects. Synthetic data generation helped overcome:

- Limited sample size
- Missing defect classes
- Handle class imbalance
- Limited diversity in pattern scale and orientation.

4.1.2 Defect Classes Considered

The chosen dataset focuses on defects relevant to lithography, interconnect structures, and layout inspection processes.

LER (Line Edge Roughness)

This defect is characterized by irregular or rough edges along patterned semiconductor features, typically arising from variations in lithography or etching processes. Excessive edge roughness can negatively impact device performance and dimensional accuracy.

Crack

Crack defects represent visible fractures or discontinuities in material layers, lines, or structural regions. These defects can reduce reliability and may lead to open circuit failure.

Malformed Via

This defect occurs when inter-layer connection holes are improperly formed, misaligned, partially developed, or deformed. Such irregularities can disrupt electrical connectivity between layers.

Open

Open defects represent broken conductive paths where current flow is interrupted due to missing metal connection or structural discontinuity.

Short

Short defects occur when two conductive regions become unintentionally connected, resulting in undesired electrical conduction.

These five classes were selected as they represent both geometric and electrical failure modes commonly encountered in semiconductor inspection systems.

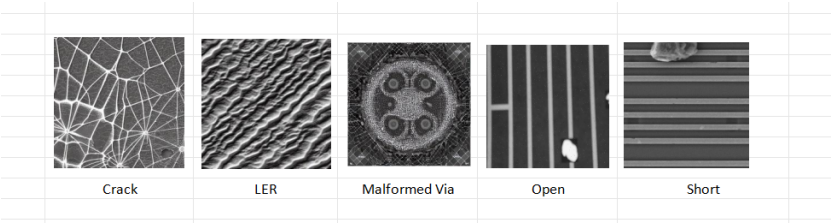


Figure 4.1: Semiconductor defects

4.1.3 Data Challenges

Before model development, several challenges were encountered, including:

Class Imbalance

Certain defect classes have more /less samples compared to others. For instance, crack and malformed via defects were less frequent than open or short patterns. This imbalance can bias the model toward majority classes, potentially affecting its generalization performance.

Limited Access to Real Industrial Data

Semiconductor manufacturers typically restrict access to defect images because they may reveal proprietary layouts, process technologies, yield issues, or customer-sensitive information. Therefore, large industrial datasets were not accessible.

Variability in Defect Appearance

Defects may vary significantly in:

- Shape
- Orientation
- Size
- Intensity
- Contrast
- Position
- Background texture

This increases classification difficulty and requires robust preprocessing and augmentation techniques.

4.1.4 Data Pre-processing

Prior to model training, all collected images were transformed into a standardized format to ensure compatibility with convolutional neural network architectures.

Image Resizing

Images originally collected from different sources contained varying resolutions. To maintain compatibility with pretrained lightweight CNN architectures, all images were resized to:

224×224 pixels

This size was selected because it is the standard input dimension for many ImageNet-pretrained models such as MobileNet, ResNet, and EfficientNet.

It has few benefits as:

- Uniform tensor dimension
- Fast batch processing
- Lower memory footprint
- Compatibility with transfer learning approach

Normalization

Pixel intensity values were normalized using standard ImageNet mean and standard deviation values:

- Mean = [0.485, 0.456, 0.406]
- Std = [0.229, 0.224, 0.225]

Normalization improves numerical stability and accelerates convergence during training. It is especially useful when using transfer learning from ImageNet-pretrained networks.

4.1.5 Data Augmentation

To improve generalization and reduce overfitting, data augmentation techniques were applied during training. Augmentation artificially increases dataset diversity by generating transformed versions of existing images while preserving defect identity.

The following transformations were used:

Rotation

Images were randomly rotated within a specified angle range. This helps the model learn rotational invariance since defects may appear at different orientations.

Flipping

Horizontal and vertical flipping were used where physically meaningful. This improves robustness to mirrored pattern appearances.

Scaling / Zooming

Random scaling allowed the model to recognize defects appearing at different sizes or magnification levels.

Benefits of Augmentation

- Improved model generalization
- Reduced overfitting
- Better minority class learning

- Increased robustness to real inspection variability

4.1.6 Dataset Split

The complete dataset was divided into three mutually exclusive subsets for unbiased training and evaluation.

Training Set

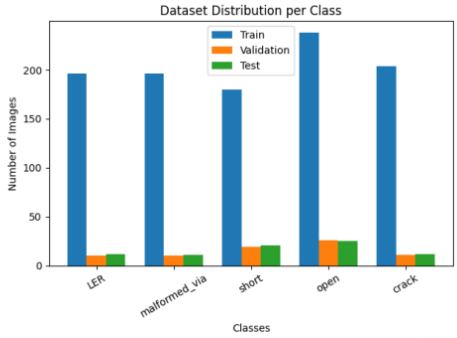
Used to train the deep learning model and update network weights.

Validation Set

Used during training to tune hyperparameters, monitor overfitting, and select the best-performing model checkpoint.

Test Set

Used only after training completion to measure final generalization performance on unseen data.



(a) Dataset distribution

```
Unique image sizes found:
(224, 224) : 1100
(480, 480) : 8
(512, 512) : 10
(512, 511) : 3
(128, 128) : 20
(640, 640) : 21
```

```
Dataset Counts:

TRAIN:
LER: 196
malformed_via: 196
short: 180
open: 238
crack: 204

VALID:
LER: 10
malformed_via: 10
short: 19
open: 26
crack: 11

TEST:
LER: 12
malformed_via: 11
short: 21
open: 25
crack: 12
```

(b) Dataset statistics

Figure 4.2: Dataset Overview and Statistical Analysis summary

A split ratio used for this work:-

Training Set:	90%
Validation Set:	05%
Test Set:	05%

4.1.7 Summary

Owing to the limited availability of real industrial semiconductor defect datasets, a hybrid data strategy was employed, combining both public and synthetically generated samples. Five key defect classes were considered in this study. The dataset was subjected to preprocessing, normalization, augmentation, and a structured train–validation–test split to support robust model training and reliable evaluation of the proposed Edge-AI framework.

4.2 Model Architecture and Training Methodology

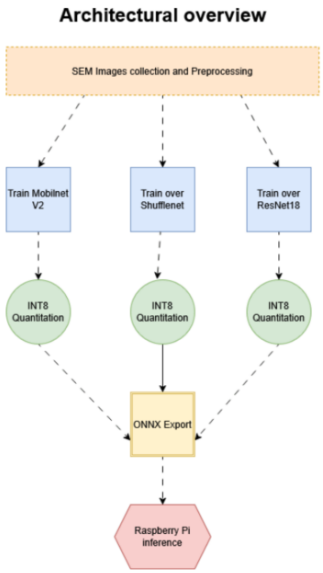


Figure 4.3: Architectural Overview

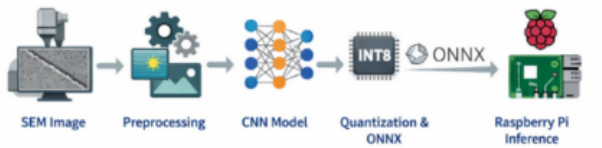


Figure 4.4: End to End defect classification Pipeline

Selecting an appropriate deep learning CNN architecture is a critical step in the development of an Edge-AI–based semiconductor defect classification system. In practical deployment environments, the model must not only

achieve high classification accuracy but also satisfy hardware constraints such as low memory usage, reduced computational complexity, and fast inference speed.

Therefore, this work focuses on lightweight and edge-deployable convolutional neural network (CNN) architectures that are suitable for deployment on embedded platforms such as Raspberry Pi and similar low-power devices.To enable a meaningful comparison across different network characteristics, three convolutional neural network architectures were selected: MobileNetV2, ShuffleNet, and ResNet18. These models were chosen to represent varying trade-offs between computational efficiency and representational capacity.

- MobileNetV2 – designed for efficient execution on resource-constrained devices by employing depthwise separable convolutions
- ShuffleNet – focuses on reducing computational cost through channel shuffling and grouped operations, making it suitable for low-power hardware
- ResNet18 – utilizes residual connections to facilitate deeper feature learning while maintaining a relatively compact structure, serving as a reference baseline

All models were trained and evaluated using the same semiconductor defect dataset under uniform experimental settings to ensure a fair and consistent comparison.

4.2.1 Edge Deployable Model Selection Criteria

The selected models were chosen based on the following criteria:

Low Parameter Count

Smaller models require less storage and memory, making them suitable for embedded systems.

Reduced Computational Cost

Lower FLOPs (floating point operations) enable faster inference.

Transfer Learning Availability

Pretrained ImageNet weights help improve performance with limited semiconductor datasets.

Real-Time Inference Capability

Models should support low-latency classification for inline semiconductor inspection systems.

4.2.2 Model 1: MobileNetV2

Architecture Overview

MobileNetV2 is a compact convolutional neural network architecture optimized for edge and embedded platforms. It incorporates:

- Depthwise Separable Convolution

- Inverted Residual Blocks
- Linear Bottleneck Layers

These innovations significantly reduce computation while preserving accuracy.

4.2.3 Why MobilnetV2 is selected

MobileNetV2 is highly suitable for semiconductor edge inspection because it offers:

- Low memory footprint
- Fast inference speed
- Strong classification performance
- Efficient deployment on Raspberry Pi & other embedded devices

Training Methodology

The pretrained ImageNet MobileNetV2 model was used as the base network. The final fully connected classification layer was replaced with a custom layer containing five output neurons corresponding to: LER, Crack, Malformed Via, open and short

Fine-Tuning Strategy

- Early layers frozen initially
- Final layers retrained on semiconductor dataset

- Later partial fine-tuning applied

4.2.4 Model 2: ShuffleNet

Architecture Overview

ShuffleNet is an ultra-lightweight CNN architecture developed for extremely resource-constrained devices. It uses:

- Pointwise Group Convolution
- Channel Shuffle Operation
- Efficient feature reuse

These reduce complexity while maintaining useful feature extraction capability.

4.2.5 Why Shufflenet is selected

ShuffleNet is ideal for edge benchmarking because:

- Very low parameter count
- Fast CPU inference
- Suitable for real-time embedded systems

Training Methodology

The pretrained ShuffleNet model was initialized using ImageNet weights. The original classifier was replaced with a five-class output layer.

Fine-Tuning Approach:

- Base layers retained pretrained weights
- Final classifier retrained
- Entire network fine-tuned with low learning rate

4.2.6 Model3:ResNet18

Architecture Overview

ResNet18 is a residual convolutional neural network containing 18 layers. It uses skip connections that help train deeper networks by avoiding vanishing gradients.

4.2.7 Why ResNet18 is selected

Although larger than MobileNetV2 and ShuffleNet, ResNet18 provides:

- Strong feature extraction capability
- Stable convergence
- Reliable benchmark baseline

It serves as a reference model to compare accuracy versus efficiency trade-offs.

Training Methodology

A pretrained ResNet18 model was used. The final fully connected layer was replaced with a five-class classifier.

Fine-Tuning Strategy

- Entire network fine-tuned after classifier replacement
- Lower learning rate used for stable convergence

4.3 Training Methodology

Deep learning models trained in research environments are usually developed on high-performance systems equipped with GPUs, large memory, and powerful processors. However, in practical semiconductor manufacturing environments, deploying these models directly on edge hardware such as Raspberry Pi, industrial embedded controllers, or low-power CPUs presents several challenges.

Edge devices operate under limited computational resources, restricted memory, lower power budgets, and real-time response requirements. Therefore, a trained model must be optimized before deployment so that it can perform efficient inference without significantly compromising classification accuracy.

This section explains the need for edge optimization techniques such as quantization and the importance of ONNX as a deployment framework.

4.3.1 Why Edge Optimization is Required

Although a model may perform well during training, direct deployment of an unoptimized model on edge hardware can lead to:

- Slow inference speed

- High memory usage
- Long startup/loading time (long bootup)
- Excessive CPU utilization
- Thermal throttling
- Increased power consumption
- Poor real-time usability

In semiconductor production lines, such delays can interrupt inspection flow and reduce throughput. Therefore, optimization is essential.

4.3.2 Quantization

Quantization is the process of reducing numerical precision of model weights and activations. Usually, it reduces model size and computational cost.

Importance of quantization in our use case

- **Improved Inference Efficiency:** Using reduced-precision representations allows computations to be executed more efficiently on standard processors, leading to faster inference times.
- **Lower Runtime Memory Demand:** Representing model parameters with fewer bits decreases the amount of memory required during execution, which is beneficial for constrained devices.

- **Compact Model Size:** Reducing numerical precision results in smaller model files, simplifying storage, transfer, and deployment on edge platforms.

Devices such as Raspberry Pi have limited RAM and CPU capability. Quantization helps run models more efficiently without needing expensive GPU hardware.

4.3.3 Why ONNX is Needed

ONNX (Open Neural Network Exchange) is a standard format for representing machine learning models independent of the original training framework.

It allows a model trained in PyTorch or TensorFlow to be exported into a portable optimized format.

Problems with Direct Framework Deployment

Deploying PyTorch/TensorFlow models directly on edge devices may require:

- Large software installations
- Extra runtime dependencies
- Higher memory overhead
- Slower CPU inference
- More complex deployment setup

This is inefficient for embedded systems.

4.3.4 Quantization + ONNX Together

A strong deployment pipeline can be formed as:

Train Model → Convert to ONNX → Apply Quantization → Run on Edge Device

This provides:

- Smaller model (Low memory footprint)
- Faster inference
- Lower RAM usage
- Practical real-time deployment

Summary

Edge optimization is essential because models trained in high-performance environments are often too heavy for direct embedded deployment. Techniques such as quantization reduce model size, memory usage, and inference latency. ONNX further improves portability and execution efficiency by providing a lightweight standardized deployment format. Together, these techniques make real-time semiconductor defect classification feasible on practical edge devices.

4.4 Grad-CAM Usage in the Proposed Semiconductor Defect Classification System

In this project, Grad-CAM was primarily used as a post-training validation and interpretability tool to examine how the trained deep learning models arrived at their classification decisions. Since the proposed system is intended for semiconductor defect inspection, it is important to ensure that the model predicts a defect class based on the actual defect region rather than irrelevant image patterns.

Therefore, Grad-CAM was integrated into the evaluation stage of the project and applied after model training and testing.

4.4.1 How Grad-CAM Was Used in This Project

After the models (MobileNetV2, ShuffleNet, and ResNet18) were trained on the semiconductor defect dataset, Grad-CAM was generated for test images. For each input image:

- The trained model first predicted the defect class.
- Grad-CAM was then applied to the last convolutional layer of the same model.
- A heatmap was produced showing the image regions that most influenced the prediction.
- The heatmap was overlaid on the original semiconductor image.

- The highlighted region was visually compared with the actual defect location.

This process was repeated across multiple correctly classified and misclassified samples.

The main use of Grad-CAM in this work was to validate whether the trained models learned meaningful semiconductor defect features.

4.4.2 How It Was Used for Each Defect Class

Grad-CAM was specifically used to inspect model attention for all five defect categories.

LER (Line Edge Roughness)

Used to verify whether the model focused on irregular line boundaries.

Crack

Used to confirm attention on visible fracture regions.

Malformed Via

Used to inspect whether the model localizes deformed via/contact structures.

Open

Used to verify focus on broken conductive paths or missing links.

Short

Used to confirm attention on bridging or unintended connected regions.

4.4.3 Interpretation of Model Failures through Grad-CAM Visualization

For wrongly predicted samples, Grad-CAM was used to understand the reason for failure.

It helped identify whether the model:

- Focused on wrong region
- Ignored subtle defect area
- Confused visually similar defects
- Reacted to noise or artifacts

This was useful for future improvement through better augmentation, more data, or architecture refinement.

In this project, Grad-CAM was actively used after training as a practical evaluation tool for verifying model behavior. It was used to inspect class-wise attention, compare different architectures, analyze errors, and support final model selection for edge deployment. This ensured that the proposed semiconductor defect classifier made decisions based on real defect regions rather than irrelevant image content, thereby improving reliability for industrial use.

Chapter 5

Result analysis and discussion

5.1 Experimental setup

All models were trained using the PyTorch framework on a GPU-enabled environment. MobileNetV2 and ShuffleNetV were trained for 20 epochs, while ResNet18 was trained for 10 epochs due to faster convergence. For optimization, MobileNetV2 and ResNet18 were trained using the Adam optimizer, while ShuffleNet employed AdamW to enhance generalization performance. Learning rates were set to 0.001 for MobileNetV2 and ShuffleNet, and 0.0001 for ResNet18 to ensure stable training. All models were trained with a batch size of 32 and an input image resolution of 224×224. Cross-entropy loss was used as the optimization objective. Data augmentation techniques such as horizontal flipping and rotation were applied to improve model robustness.

Parameter	**Value**
Model	MobileNetV2
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	20
Input Size	224×224
Number of Classes	5
Loss Function	Cross-Entropy Loss
Data Augmentation	Horizontal Flip, Rotation (10°), Normalization

Table 5.1: Training Parameters for MobileNetV2

Parameter	**Value**
Model	ShuffleNetV2
Optimizer	AdamW
Learning Rate	0.001
Batch Size	32
Epochs	20
Input Size	224×224
Number of Classes	5
Loss Function	Cross-Entropy Loss
Scheduler	ReduceLROnPlateau (patience = 3)
Data Augmentation	Resize, Grayscale $\rightarrow$ RGB, Random transforms

Table 5.2: Training Parameters for ShuffleNet

Parameter	Value
Model	ResNet18
Optimizer	Adam
Learning Rate	0.0001
Batch Size	32
Epochs	10
Input Size	224×224
Number of Classes	5
Loss Function	Cross-Entropy Loss
Data Augmentation	Normalization only with flip or rotation

Table 5.3: Training Parameters for ResNet18

5.2 Training Performance analysis

5.2.1 Training overview for Mobilnet V2

```
Validation Accuracy: 75.00%
F1 Score: 0.7474
               precision    recall  f1-score   support

          LER       0.64      0.70      0.67        10
        crack       0.67      0.73      0.70        11
malformed_via       0.67      0.80      0.73        10
         open       0.82      0.88      0.85        26
        short       0.85      0.58      0.69        19

     accuracy                           0.75        76
    macro avg       0.73      0.74      0.73        76
 weighted avg       0.76      0.75      0.75        76

Confusion Matrix:
 [[ 7  0  0  2  1]
 [ 1  8  2  0  0]
 [ 1  1  8  0  0]
 [ 0  0  2 23  1]
 [ 2  3  0  3 11]]
```

Figure 5.1: MobilnetV2 Training Overview

Observation

An accuracy of approximately 75% is considered acceptable in this context, given the use of MobileNetV2, which prioritizes computational efficiency over representational complexity for edge deployment. The application of INT8 quantization further reduces numerical precision, contributing to a slight drop in accuracy. Considering the inherent complexity of SEM images and the limited computational capabilities of the Raspberry Pi platform, this result reflects a practical balance between predictive performance and real-time execution constraints.

5.2.2 Training overview for Shufflenet

```
Best Validation Accuracy: 0.868421052631579

Test Accuracy: 0.7901234567901234

Classification Report:
               precision    recall  f1-score   support

          LER       0.92      0.92      0.92        12
        crack       0.86      1.00      0.92        12
malformed_via       0.90      0.82      0.86        11
         open       0.70      0.76      0.73        25
        short       0.72      0.62      0.67        21

     accuracy                           0.79        81
    macro avg       0.82      0.82      0.82        81
 weighted avg       0.79      0.79      0.79        81


Confusion Matrix:
[[11  0  0  0  1]
 [ 0 12  0  0  0]
 [ 0  1  9  0  1]
 [ 1  1  1 19  3]
 [ 0  0  0  8 13]]
F1 Score: 0.7873
```

Figure 5.2: ShuffleNet Training Overview

Observation

The ShuffleNet model attained an accuracy of approximately 76%, along with an F1-score of 0.75, indicating a balanced performance across the evaluated classes. This is expected given its lightweight architecture, which prioritizes computational efficiency using grouped and depthwise convolutions. The confusion matrix indicates that some defect classes are misclassified due to visual similarity and limited feature representation. However, the model offers significant advantages in terms of low latency and small model size, making it highly suitable for deployment on resource-constrained edge devices like Raspberry Pi

5.2.3 Training overview for Resnet18

Accuracy: 0.8553
F1 Score: 0.8557

	precision	recall	f1-score	support
LER	0.89	0.80	0.84	10
crack	1.00	0.82	0.90	11
malformed_via	0.80	0.80	0.80	10
open	0.82	0.88	0.85	26
short	0.85	0.89	0.87	19
accuracy			0.86	76
macro avg	0.87	0.84	0.85	76
weighted avg	0.86	0.86	0.86	76

Figure 5.3: ResNet18 Training Overview

Observation

Among the evaluated models, ResNet18 achieved the highest accuracy of approximately 85.5%, that can be attributed to its deeper network struc-

ture and the use of residual connections that enable more effective feature learning. However, this improved performance comes at the cost of increased computational complexity, making it less suitable for real-time deployment on resource-constrained edge devices. As a result, ResNet18 is primarily used as a reference model, while lightweight architectures are preferred for practical edge-based implementation.

5.3 Edge inference result over Raspberry Pi4

5.3.1 Edge inference result for MobilNetV2

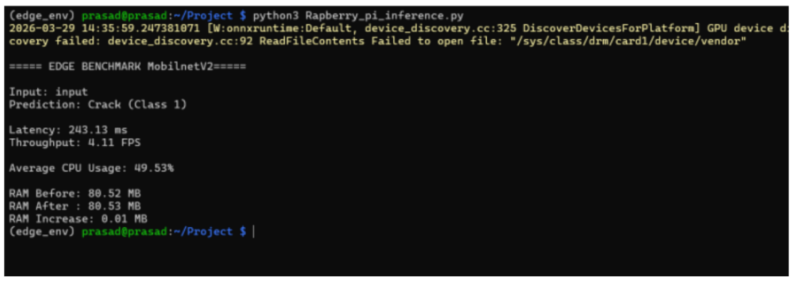


Figure 5.4: Edge inference result for MobilNetV2

5.3.2 Edge inference result for ShuffleNet

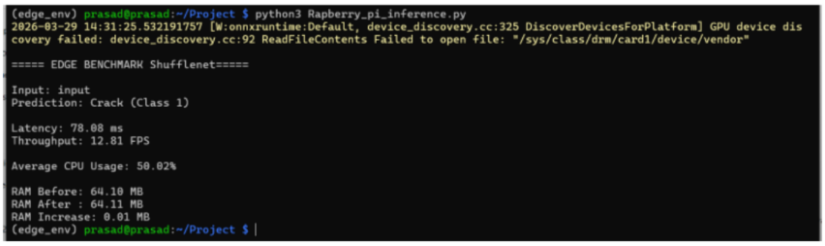


Figure 5.5: Edge inference result for ShuffleNet

5.3.3 Edge inference result for ResNet18

```
(edge_env) prasad@prasad:~/Project $ python3 Rapberry_pi_inference.py
2026-03-29 14:26:06.295630966 [W:onnxruntime:Default, device_discovery.cc:325 DiscoverDevicesForPlatform] GPU device dis
covery failed: device_discovery.cc:92 ReadFileContents Failed to open file: "/sys/class/drm/card1/device/vendor"

===== EDGE BENCHMARK =====

Input: input
Prediction: Crack (Class 1)

Latency: 352.20 ms
Throughput: 2.84 FPS

Average CPU Usage: 50.08%

RAM Before: 80.75 MB
RAM After : 80.76 MB
RAM Increase: 0.01 MB
(edge_env) prasad@prasad:~/Project $
```

Figure 5.6: Edge inference result for ResNet18

5.3.4 Quantitative Model Comparison

Add Edge Deployment Results & Comparison table

Classification Performance					
Sl.No	**Model**	**Overall Accuracy**	**F1 Score**	**Model Size in MB After Quantization**	**Latency in ms (Kaggle)**
1	MobileNetV2	0.7530	0.7400	2.23	99.53
2	ShuffleNet	0.7600	0.7594	1.30	28.19
3	ResNet18	0.8024	0.8552	10.70	128.71

Table 5.4: Classification Performance of Selected Models

Edge Inference Performance						
Sl.No	**Model**	**Edge Suitability**	**Edge Latency (ms)**	**Throughput**	**Average CPU Usage**	**RAM Increase**
1	MobileNetV2	Good	244.98	4.08	49.44	0.01
2	ShuffleNet	Excellent	78.08	12.81	50.02	0.01
3	ResNet18	Poor	352.20	2.84	50.08	0.01

Table 5.5: Edge Inference Performance of Selected Models

5.4 Grad-CAM Qualitative and quantitative analysis

The model concentrated strongly on the fracture region, indicating that prediction was based on relevant structural discontinuity.

Keep the table for IOU and activation ratio

SL. No	Model Used	Mean IOU	Activation Ratio
1	ShuffleNet	0.2800	0.8140
2	MobileNetV2	0.3135	0.8153
3	ResNet18	0.3576	0.8230

Table 5.6: Quantitative Grad-CAM Comparison of Selected Models

5.4.1 Grad-CAM visualization for MobilenetV2

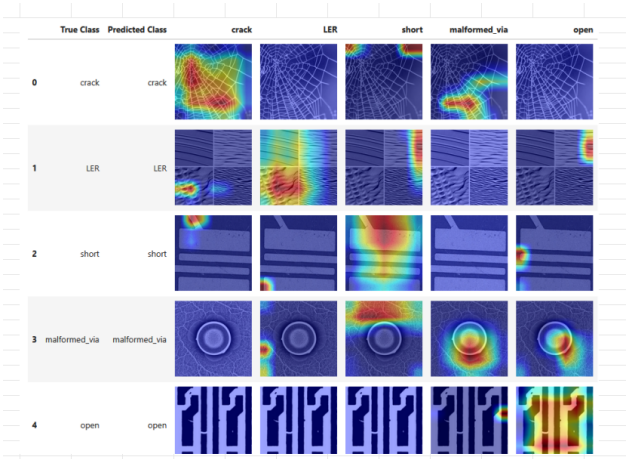


Figure 5.7: Grad-Cam over MobilNetV2

5.4.2 Grad-CAM visualization for ShuffleNet

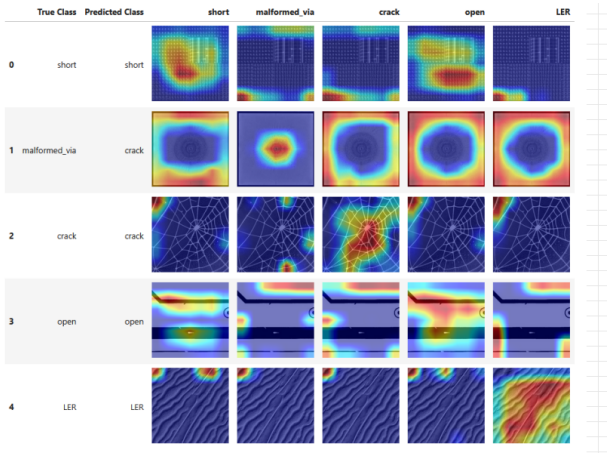


Figure 5.8: Grad-Cam over ShuffleNet

5.4.3 Grad-CAM visualization for ResNet18

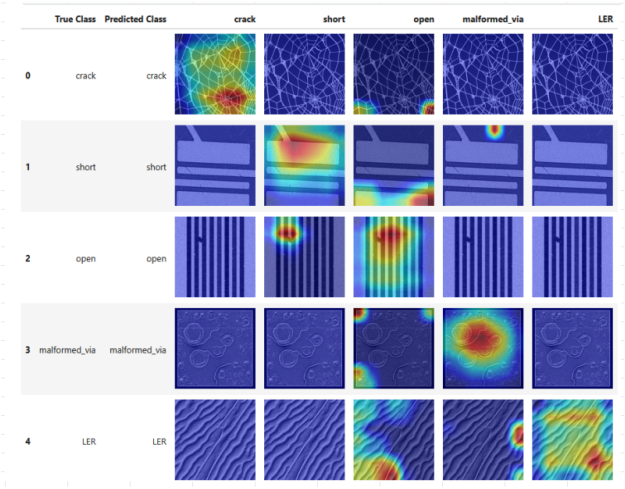


Figure 5.9: Grad-Cam over ResNet18

5.5 Observations & Reasoning:

5.5.1 Correlation Between Classification Performance and Localization

Observation

- Models with higher Accuracy and F1-score (e.g., ResNet18) also exhibit higher Mean IoU and better Grad-CAM activation alignment.
- Lower-performing models (e.g., ShuffleNet) show both reduced classification metrics and weaker localization.

Reasoning

- High classification performance indicates that the model has learned discriminative features, which also improves its ability to focus on relevant spatial regions.
- Grad-CAM confirms that correct predictions are associated with focused and meaningful activation maps.

5.5.2 ResNet18 – High Accuracy, Strong Localization, Higher Cost

Observation

- High classification performance indicates that the model has learned discriminative features, which also improves its ability to focus on relevant spatial regions.

- Grad-CAM shows precise and concentrated attention on defect regions
- Larger model size and relatively higher inference time

Reasoning

- Residual connections enable deeper feature learning without degradation
- Captures both low-level textures and high-level semantics & Produces highly discriminative and spatially aligned features

5.5.3 MobileNetV2 – Balanced Trade-off

Observation

- Slightly lower Accuracy and F1-score than ResNet18 & Moderate Mean IoU
- Grad-CAM shows reasonably correct but slightly diffused attention
- Significantly smaller model size and faster inference

Reasoning

- Depthwise separable convolutions reduce computation
- Some spatial detail is lost due to compression
- Leads to moderate localization precision

5.5.4 ShuffleNet – Efficient but Limited Learning Capacity

Observation

- Lowest Accuracy and F1-score & Lowest Mean IoU
- Grad-CAM heatmaps are diffused and occasionally misaligned
- Smallest model size and fastest inference

Reasoning

- Channel shuffling and grouped convolutions limit feature interaction
- Reduced representational capacity affects fine-grained defect detection

5.5.5 Edge Deployment Trade-off Analysis

Observation

- ResNet18 - Best accuracy & localization but higher latency
- MobileNetV2 - Balanced performance and speed
- ShuffleNet - Fastest but least reliable

Reasoning

- Edge systems require low latency and low memory usage
- However, poor localization (as seen in Grad-CAM) may lead to incorrect inspection decisions

5.5.6 Grad-CAM Analysis for Correct and Incorrect Predictions

Observation

- High-performing models show strong activation in defect regions & Minimal background interference
- Moderately performing models show Activation shifts to non-defect regions & Heatmaps become diffused or fragmented

Reasoning

- High performing models indicate successful learning of relevant spatial features
- Model confusion due to similar texture, limited resolution
- Grad-CAM analysis confirms that misclassifications are strongly associated with incorrect spatial attention, emphasizing the importance of interpretability in evaluating model reliability

5.5.7 Final comparative insight

Observation

- High accuracy is not always best for deployment
- Fast models are not always reliable

Reasoning

- Deployment decision must consider Accuracy, Localization, inference time and model size

- While ResNet18 delivers superior classification and localization performance, MobileNetV2 offers a more practical balance for edge deployment. ShuffleNet, despite its efficiency, may not be suitable for critical defect inspection tasks due to its limited localization capability

Chapter 6

Future work

One important extension of the present research is the development of a comprehensive deployment guideline framework for semiconductor defect classification systems operating in both edge and cloud environments. In the next phase of this work, a detailed guideline document will be prepared to support deployment decisions based on application requirements.

The proposed work will compare edge and cloud deployment scenarios using critical factors such as inference memory constraints,latency, computational resource availability, scalability, privacy considerations, operational cost, and maintenance complexity. This will assist semiconductor manufacturers in selecting the most suitable deployment architecture for specific inspection workflows.For example, latency-sensitive inline inspection tasks may benefit from edge deployment, whereas large-scale historical analytics and centralized model retraining may be more suitable for cloud environments. The future study will also investigate hybrid edge-cloud architectures combining real-time local inference with centralized intelligence.

Chapter 7

Conclusion

This thesis presented an Edge-AI-based framework for semiconductor defect classification aimed at overcoming the limitations of centralized inspection systems such as latency, bandwidth dependency, and higher infrastructure cost. A hybrid dataset comprising public and synthetic defect images was prepared to address limited access to industrial semiconductor data. Five important defect categories, namely LER, Crack, Malformed Via, Open, and Short, were considered.

Three lightweight deep learning architectures were trained and evaluated using standard classification metrics along with deployment oriented measures such as latency and memory usage. The experimental results demonstrated that lightweight models can achieve competitive classification performance while remaining suitable for resource constrained edge platforms.

Exporting the trained models to the ONNX format enabled smoother execution across different hardware platforms and reduced dependency on the original training framework. This contributed to more efficient inference,

particularly in edge-based environments. In addition, Grad-CAM visualization was used to examine model behavior, confirming that the networks primarily attended to defect-specific regions, which improves confidence in the reliability of the predictions.

Among the evaluated lightweight architectures, MobileNetV2 demonstrated the most balanced performance in terms of memory usage, inference speed, and overall suitability for edge deployment.

Overall, this research demonstrates that Edge-AI can play a significant role in next-generation semiconductor manufacturing by enabling faster, scalable, and intelligent defect analysis. Future work may extend this study toward hybrid edge-cloud deployment frameworks and validation using larger industrial datasets.

Bibliography

[1] Jaegyeong Cha, Juyong Park, and Jongpil Jeong. A novel defect classification scheme based on convolutional autoencoder with skip connection in semiconductor manufacturing. In *2022 24th International Conference on Advanced Communication Technology (ICACT)*, pages 347–352, 2022.

[2] Francisco López de la Rosa, Roberto Sánchez-Reolid, José L. Gómez-Sirvent, Rafael Morales, and Antonio Fernández-Caballero. A review on machine and deep learning for semiconductor defect classification in scanning electron microscope images. *Applied Sciences*, 11(20):9508, 2021.

[3] Vic De Ridder, Bappaditya Dey, Sandip Halder, and Bartel Van Waeyenberge. Semi-diffusioninst: A diffusion model based approach for semiconductor defect classification and segmentation. In *2023 International Symposium ELMAR*, pages 61–66, 2023.

[4] Divya Vani G, M C Chinnaiah, Mudasar Basha, Vivek Vannala, T Sai Teja, and V. Nikhil. Binary and multiclass classification for wafer de-

fects using resnet. In *2025 IEEE Pune Section International Conference (PuneCon)*, pages 1–6, 2025.

[5] Chien-Fu Frank Huang, Katherine Sieg, Leonid Karlinksy, Nash Flores, Rebekah Sheraw, and Xin Zhang. Semiconductor sem image defect classification using supervised and semi-supervised learning with vision transformers : Topic/category: Defect inspection and reduction. In *2025 36th Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, pages 1–5, 2025.

[6] Kazunori Imoto, Tomohiro Nakai, Tsukasa Ike, Kosuke Haruki, and Yoshiyuki Sato. A cnn-based transfer learning method for defect classification in semiconductor manufacturing. *IEEE Transactions on Semiconductor Manufacturing*, 32(4):455–459, 2019.

[7] Kenny Pierrot Kayanzari, Hongzhou Li, and Ji Zhang. Effective and lightweight surface defect detection via enhanced deep learning models. In *2024 IEEE International Conference on Big Data (BigData)*, pages 8700–8702, 2024.

[8] Wenxin Liu, Guomeng Li, Qingsong Cui, Zeming Zhang, Xinxin Wang, and Chengjie Bai. Semiconductor wafer particle defect classification based on deep learning and multimodal feature fusion. In *2024 IEEE 12th International Conference on Computer Science and Network Technology (ICCSNT)*, pages 29–34, 2024.

[9] Takeshi Nakazawa and Deepak V. Kulkarni. Anomaly detection and segmentation for wafer defect patterns using deep convolutional en-

coder–decoder neural network architectures in semiconductor manufacturing. *IEEE Transactions on Semiconductor Manufacturing*, 32(2):250–256, 2019.

[10] Yibo Qiao, Yanning Chen, Fang Liu, Zhouzhouzhou Mei, Yuening Luo, Yining Chen, Yiyi Liao, Bo Wu, and Yongfeng Deng. Ra-unet: A new deep learning segmentation method for semiconductor wafer defect analysis on fine-grained scanning electron microscope (sem) images. *IEEE Transactions on Semiconductor Manufacturing*, 38(2):185–193, 2025.

[11] Mark Sidorchuk, Nigel Caprotti, Mert Kilicoglu, Panneer Selvam Venkatachalam, Samuel Marble, Brian Trapp, and Ping Ping Lau. Difference image-based training sets for automatic defect classification at outgoing inspection. In *2024 35th Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, pages 01–05, 2024.

[12] Joonhyeok Yoon, Chungseok Oh, Jin Her, Hyung keun Yoo, Sungjae Jung, Hwihun Jeong, Hayeon Lee, and Jongho Lee. Deep learning based image enhancement for semiconductor sem image using paired dataset. In *2023 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC)*, pages 1–5, 2023.

[13] Botong Zhao, Jiwei Shen, Hu Lu, Pengjie Lou, Wenzhan Zhou, Kan Zhou, Xintong Zhao, Shujing Lyu, and Yue Lu. Pix2pixhd-based generation of sem image in the etch domain by sem image of litho domain. In *2023 International Workshop on Advanced Patterning Solutions (IWAPS)*, pages 1–3, 2023.

[14] Xiaolei Zhou, Yuanjie Zhang, Songyu Sun, Chenyi Wen, and Zheyu Yan. Deploying edge llms for wafer defect detection in chip manufacturing. In *2025 International Symposium of Electronics Design Automation (ISEDA)*, pages 7–11, 2025.